



CubeSat ADCS Simulator

User Manual

Version 1.8

July 2021

Table of Contents

Version history	3
1 Introduction	4
2 Minimum system requirements	4
3 License restrictions	4
4 Installation Instructions	4
5 User Interface	5
5.1 Model Window	5
5.2 Object Selection	6
5.3 Changing Object Properties	7
5.4 Simulation Run/Pause/Reset	7
5.5 Simulation Playback	8
5.6 Object State	8
5.7 Simulation Settings	9
5.8 New, Load, Save	10
5.9 Creating Animations.....	10
6 HIL simulation	11
7 Built-in Scripts	12
7.1 Standard simulation script and report.....	13
7.2 Scheduler Script	14
8 Command-line and “Headless” Mode	19
Appendix A Simulation Model Overview.....	21
Appendix B Coordinate Systems	22
Appendix C Model Object Detail.....	23
Satellite Components.....	26
HIL satellite models.....	34
References	35

Version history

Version ¹	Date	Comments
1.0	July 2017	Initial version
1.1	January 2018	Updated to reflect changes to UI (from old QB50 interface)
1.3	March 2020	• Updated UI description to include buttons on toolbar, including test script button • Updated model descriptions in appendix to reflect new method of surface modelling (using panels and solar panels) • Added section on licensing restrictions
1.4	July 2020	• Added description for Scheduler Script – to allow running of scheduled simulation sequence. • Added section on command line and headless modes
1.8	July 2021	• Updated section on scripting and command-line interface • Updated some sections to reflect CubeSpace ADCS2 functionality • Document version number updated to match current EOS engine version

¹ Document version number follows the version number of the software

1 Introduction

EOS is a modular simulation application for Earth-orbiting satellite scenarios. It makes use of an object oriented simulation configuration with a visualisation aspect.

The simulator will initialise a scenario where the ADCS (Attitude Determination and Control System) of a CubeSat is simulated. The ADCS implementation is that of the CubeSpace ADCS module². The attitude dynamics of the satellite is simulated in a specified orbit, by calculating realistic disturbances and using accurate models for the attitude actuators. The simulator provides the functionality to evaluate the performance of the CubeSpace ADCS for the given configuration, and also to evaluate the effect of different control gains and controller parameters. Note that the CubeADCS User Manual [1], [2] should be consulted in order to fully understand the various options that are available in the simulator.

It is also possible to connect the simulator to the CubeSpace CubeComputer through a USB/UART interface cable, or PCAN USB adapter (ADCS Gen2 only), in order to perform a hardware-in-the-loop (HIL) simulation in parallel.

The EOS simulator supports both 1st generation and 2nd generation CubeSpace ADCS units. The primary difference in the simulator will be in the CuibeACP (ADCS Gen1) and CubeComputer (ADCS Gen2) model objects. Both these objects fulfil the role of computing the attitude estimate and calculating control commands, however the names reflect the components as they are referred to in the CubeSpace documentation. The properties for these components are also dissimilar – see the Appendix for more detail.

2 Minimum system requirements

Windows Vista, 7, 8 or 10

20MB Hard-drive space

1GB RAM or more

Screen resolution: at least 1024 x 768

3 License restrictions

Depending on the type of license, certain functionality and settings in EOS may be limited. For “Mission” licenses, the simulator is licensed only for a particular mission. In such cases, the user will be restricted when changing the orbit, and satellite moment of inertia. Specifically, the orbit altitude can be changed with a 200km range (+/- 100 km) from the initially specified orbit, and the inclination may only be changed by +/- 5° from the initially specified inclination. The moments of inertia can be adjusted in a range for the specified satellite form factor (1U, 2U, 3U or 6U). The simulator will also only be able to load scenarios with fixed satellite component configuration.

4 Installation Instructions

Prior to running the simulator, you will need to install the following dependencies (if they are not already installed on your system):

- Microsoft XNA redistributable 4.0 package:

² www.cubespace.co.za

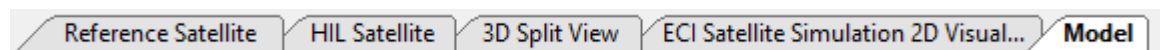
<https://www.microsoft.com/en-za/download/details.aspx?id=20914>

- Microsoft Visual C++ 2015-2019 Runtime libraries (x86 version)

The EOS simulator is contained in a ZIP file. Simply unzip the contents of the ZIP file to a convenient location on your hard drive and double-click on the EOS.exe application to start it.

5 User Interface

The program will have 5 tabs running along the top of the window. Each tab shows either a different 3D visualisation of the reference satellite and/or HIL satellite, or a 2D map of the Earth showing positions of the satellite in the orbit. Finally there is a tab called *Model*, which shows the various model objects and their dependencies.



There is also a *Model Properties* window on the right, from where individual properties of the selected model object can be adjusted.

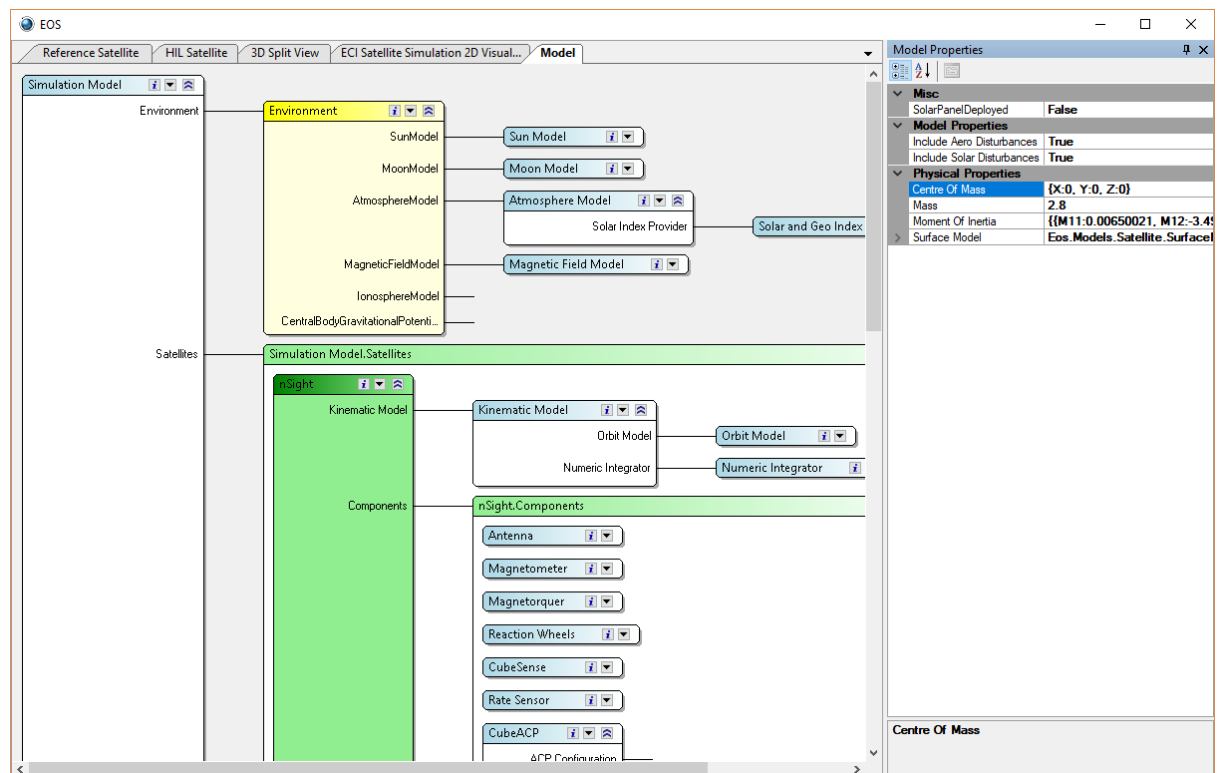


Figure 1 EOS User Interface, showing the Model tab (left) and Model Properties window (right)

5.1 Model Window

The *Model* Window shows how the simulation scenario is constructed. There is a main satellite *Simulation Model* which depends on an *Environment* Model and a number of *Satellite* Models. The entire model object tree is shown below. The appendix also describes the simulation model in more detail.

Simulation model object	Detail
Simulation Root	Root model object
Environment	Satellite environment model
Sun Model	Provides sun position in ECI frame
Moon Model	Provides moon position in ECI frame - not used in ADCS simulation
Atmosphere Model	Provides atmosphere density as a function of orbit position (in ECI coordinates). Uses NRLMSISE-00 model.
Solar and Geo Index Provider	Provides Ap indices and F10.7 solar flux to the atmosphere model. The model predicts solar conditions using mean 11-year solar cycle
Magnetic Field Model	Provides magnetic field vector as a function of orbital position (in ECI frame). Uses IGRF magnetic field model implementation.
Satellites	Container for all the satellites in the simulation
<Sat Name>	Reference satellite
Kinematic Model	Provides orbit position and velocity, and attitude angles and angular rates to the root simulation object. This model implements a numerically integrated attitude, where position and velocity is supplied by an SGP4 orbit model.
Orbit Model	SGP4 based orbit propagator. Provides satellite orbit position and velocity as a function of time
Numeric Integrator	The numeric integrator used by the attitude dynamics propagator. Uses RK45 implementation
Components	Container for all the components in the satellite
Antenna	Antenna object. Serves no simulation purpose, only visual
Magnetometer	Simulated magnetic field sensor.
Magnetorquer	Simulated magnetic torquer rods.
Reaction Wheels	Simulated 3-axis reaction wheels.
CubeSense	Simulated sun and nadir sensor
Rate Sensor	Simulated rate sensor
CubeACP	Attitude Control Processor - this represents the CubeComputer in the CubeSpace ADCS bundle
CubeStar	Simulated star sensor
CSS	Simulated coarse sun sensor
Communications Model	Not Used
<HIL Sat Name>	HIL satellite. This has the same dependencies and components as the Reference satellite, but in stead of a "software" implementation of the CubeACP, the HIL equivalent will communicate sampled sensor measurements to actual CubeComputer using USB/UART cable, and retrieve actuator commands from it.
Geographic Locations	Not Used

Figure 2 CubeSat ADCS Simulation Model Tree

5.2 Object Selection

The selected model object can be changed by selecting a new object in the *Model* window. Clicking for instance on the Orbit Model brings up the available properties in the Model Properties window.

It is also possible to select an object through one of the 3D visualisation tabs, by clicking on the relevant satellite, or visualised component. Not all model objects have a visual representation (the Kinematic Model for instance).

Finally, it is also possible to select a model object from the drop-down list on one of the visualisation tabs.



Figure 3 Different methods to select a different model object. (left) Using the Model window. (middle) By interacting with the 3D visualisation. (right) Selecting from the drop-down list

5.3 Changing Object Properties

Once an object has been selected, the properties for that model object will be displayed in the *Model Properties* window. From here it is possible to change the properties to user required values. More detail about each model block implementation and its properties is given in the appendix.

There are properties which may only be adjusted when the simulation is in the ‘Reset’ state. These are so-called *Initial Value* properties. They may be changed before the simulation has started. When the simulation has already been started, but in the ‘Paused’ state (see next section) it is also possible to change some properties of the selected model object, but only if this is allowed for the particular property (i.e. not *Initial Value* properties). Properties which cannot be altered will be shown in a “greyed-out” state.

Model Properties	Model Properties
Model Properties Include Aero Disturbances True Include Solar Disturbances True Physical Properties Centre Of Mass {X:0, Y:0, Z:0} Mass 2.8 Moment Of Inertia {{M11:0.00650021, M12:0.00000000, M13:0.00000000, M21:0.00000000, M22:0.00650021, M23:0.00000000, M31:0.00000000, M32:0.00000000, M33:0.00650021}} Surface Model Eos.Models.Satellite.Satellite ShieldingSegments (Collection) SurfaceSegments (Collection)	Model Properties Include Aero Disturbances True Include Solar Disturbances True Physical Properties Centre Of Mass {X:0, Y:0, Z:0} Mass 2.8 Moment Of Inertia {{M11:0.00650021, M12:0.00000000, M13:0.00000000, M21:0.00000000, M22:0.00650021, M23:0.00000000, M31:0.00000000, M32:0.00000000, M33:0.00650021}}

Figure 4 Properties for the Satellite Model Object during Reset state (left) and while simulation is paused (right)

5.4 Simulation Run/Pause/Reset

The bar along the top of the visualisation windows displays information about the current selected object, current simulation and playback time and allows the simulation to be controlled.

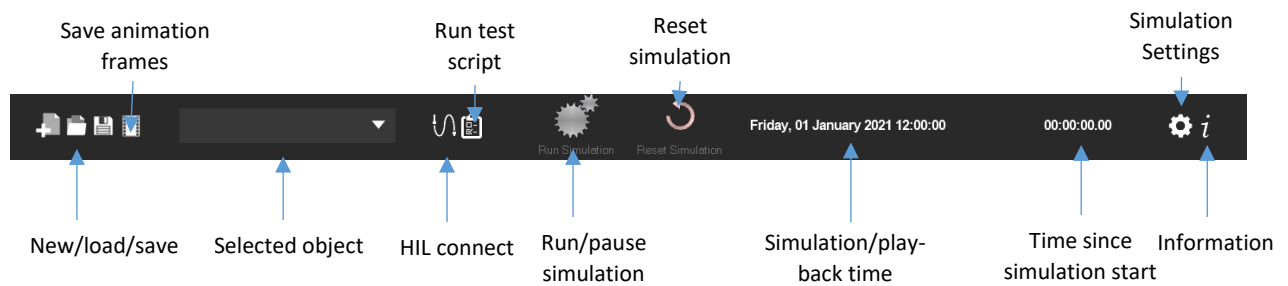


Figure 5 Simulation control toolbar

The simulation is initiated by pressing the *Run Simulation* button . The simulation can be paused by pressing the same button again.

To reset the simulation so that it starts from the beginning, press the *Reset Simulation*  button.

5.5 Simulation Playback

Once the simulation has been run, state for the model objects will be recorded and stored in a cache. This state can be used to play back the simulation up to the point where the simulation was paused. (Pressing the Reset Simulation button will also clear the cache).

The playback toolbar at the bottom of the user interface can be used to play back the simulation, or use the time slider the playback time to a specific point of interest.

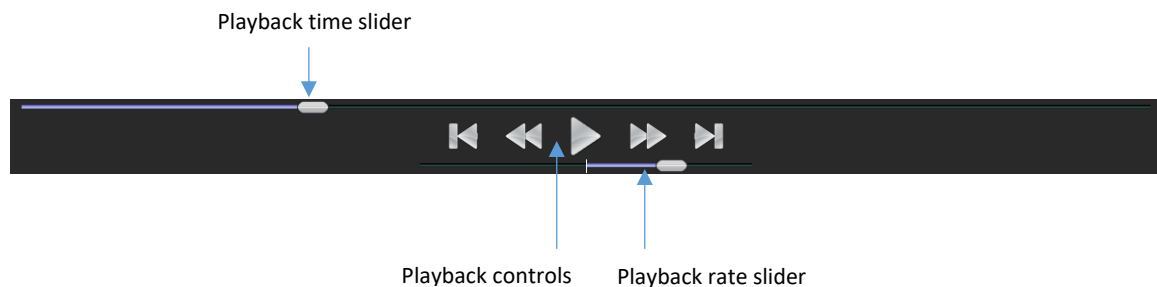


Figure 6 Playback toolbar

5.6 Object State

Once an object has been selected, the visualisation tabs will expand two additional windows – the selected object state window and a plot window.

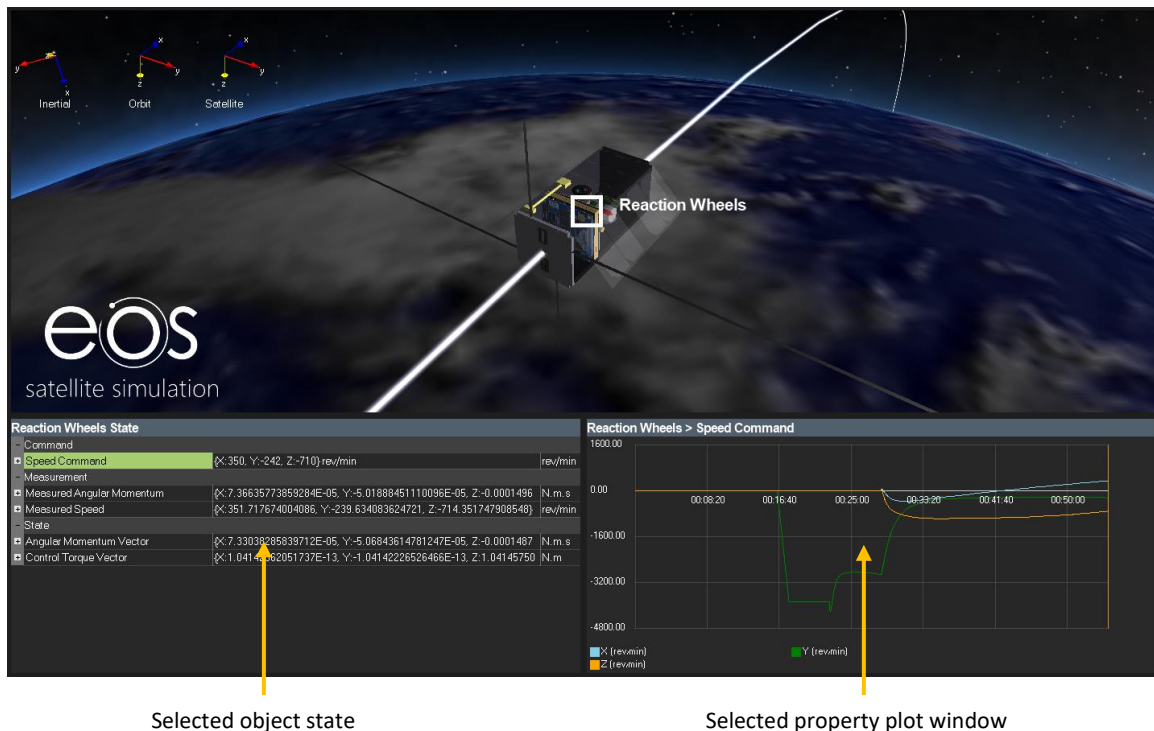


Figure 7 UI after object selection

The *Selected Object State* window will display the recorded state of the selected model object for the current playback epoch. The state is listed as a number of properties, some which can be expanded with further child properties.

A property in the *Selected Object State* window can be selected which cause a plot of that property over time to be displayed in the *Plot* window.

A number of buttons become visible when hovering over the plot window. The  button allows the data in the chart to be exported to a CSV file.

5.7 Simulation Settings

The simulation settings can be varied by pressing the  button.

Simulation settings include the *Simulation Start Epoch* and *Duration*. It is also possible to let the simulation run without a fixed duration by checking the *Run Forever* checkbox. In this case the simulation will continue to run until the user manually pauses the simulation.

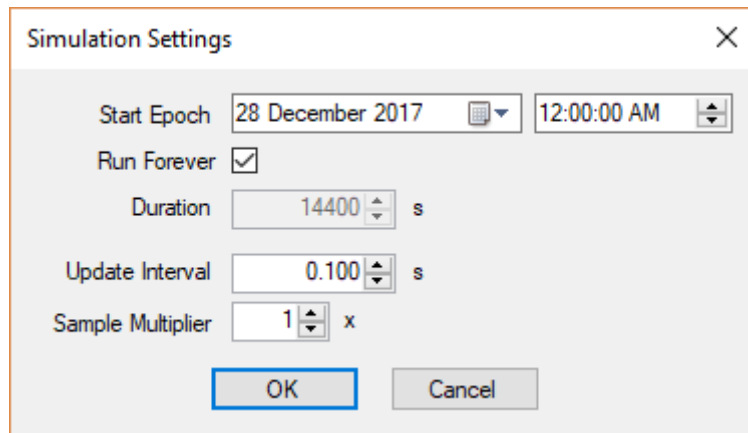


Figure 8 Simulation Settings Window

The simulation runs in fixed update intervals. The interval between simulation update iterations can be adjusted using the *Update Interval* setting.

Finally, the interval at which the simulation state is stored to cache can be controlled using the *Sample Multiplier* setting. If this setting is set to 1, the simulation state will be stored at every update interval. If it is set to 2, it will only be stored every other update interval and so on. This setting allows memory to be conserved for simulations with a long time span.

The simulation settings can only be adjusted when the simulation is in the *Reset* state.

5.8 New, Load, Save

The buttons to start a new simulation, load an existing simulation configuration, or save the current configuration are found at the top, left of the visualisation windows.



Figure 9 New, Load and Save buttons

Pressing the 'New Simulation' button will initialise the simulation using the default configuration. The project file that is created when pressing the 'Save' button is a text file which captures all the current property values of the objects in the simulation, as well as the simulation settings (simulation start time, interval, etc.). This file can be read back at a later stage using the 'Load' button.

5.9 Creating Animations

EOS does currently not have the ability to save animations directly to a video file. It can be used to output a number of image frames, which can be converted into an animation using third-party

applications. The 'Save Animation Frames' button, , will open a dialog window from which a number of settings is available.

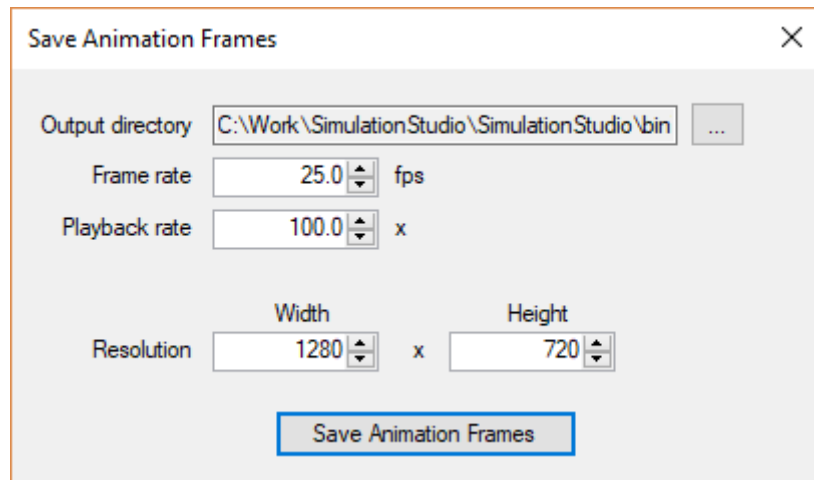


Figure 10 Save Animation Frames window

The destination folder in which image frames will be saved can be changed by pressing the  button. This should ideally be an empty folder. Images will be saved with filenames frameXXXXX.jpg, where XXXXX is the number of the frame, starting at 00000.

The amount of simulation time between consecutive image frames is controlled using the *Frame Rate* and *Playback rate* settings. Typical video files have frame rates of 25 frames per second. A playback rate of 1.0x will result in an animation that reflects real time.

The dimensions of the output images are adjusted using the *Resolution Width* and *Height* settings. Once settings have been adjusted to desired values, use the 'Save Animation Frames' button to start saving frames.

6 HIL simulation

In order to perform a Hardware-in-the-Loop (HIL) simulation, the computer running the simulation has to be connected to the CubeComputer in the ADCS module using the UART-USB GSE cable supplied with the ADCS hardware.

In the case of a Gen2 ADCS unit, it is also possible to connect using a PCAN USB adapter. The HIL interface (PCAN or UART) has to be selected from the CubeComputer model object, for a Gen2 ADCS simulation prior to attempting the UART connection. The 1st generation ADCS units do not have this option and can only accept a UART connection for HIL testing.

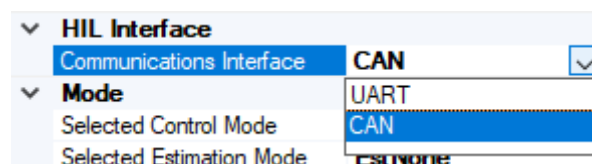


Figure 11 ADCS Gen2 CubeComputer properties, showing HIL connection options



A simulation is initiated by pressing the connect, , button. The connection status (or possibly a connection error message) is displayed at the bottom of the HIL visualisation window.

The HIL simulation will simulate two satellites at the same time – the reference satellite (which is the same one being simulated in the absence of a HIL connection) and a HIL satellite. Both satellites will have exactly the same position, but their attitude dynamics are simulated separately.

It should be noted that even though the implementation of the ADCS estimation and control algorithms are matched in the CubeComputer control program, and EOS reference implementation, the two satellites may display slight differences in behaviour over time. This is due to a combination of random sensor noise and number rounding that may differ between the two implementations.

7 Built-in Scripts

EOS has the ability to execute built-in scripts, to verify certain ADCS functionality in an automated

fashion. The built-in scripts are accessed by pressing the Run Test Script, , button. This will open up a window with available scripts to run. The Scheduler and Standard Verification scripts are included by default.

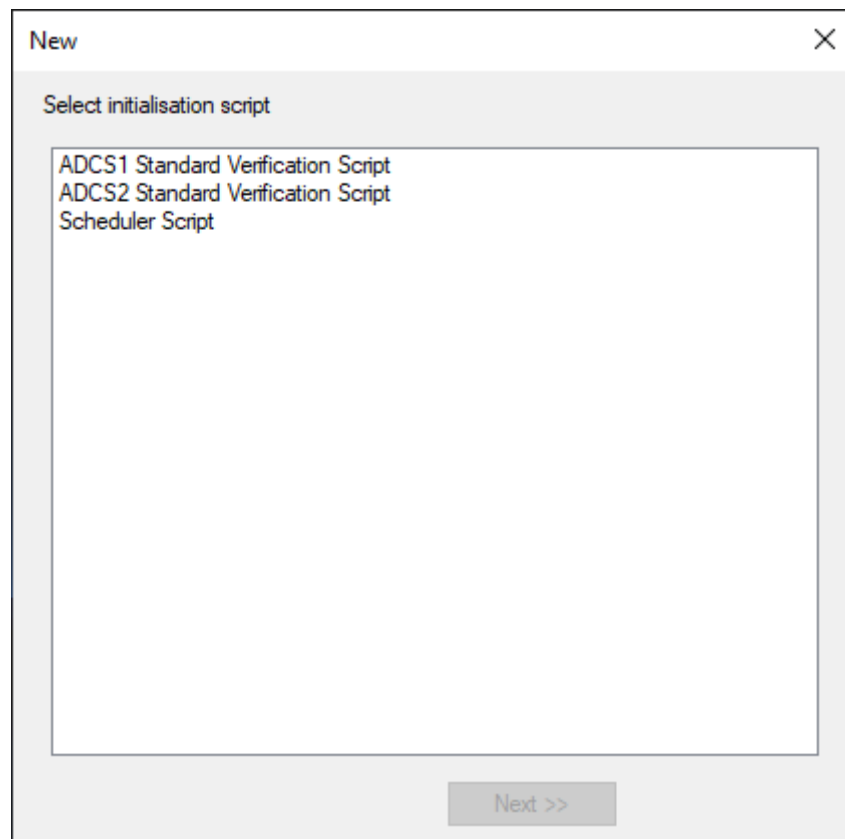


Figure 12 Script selection window

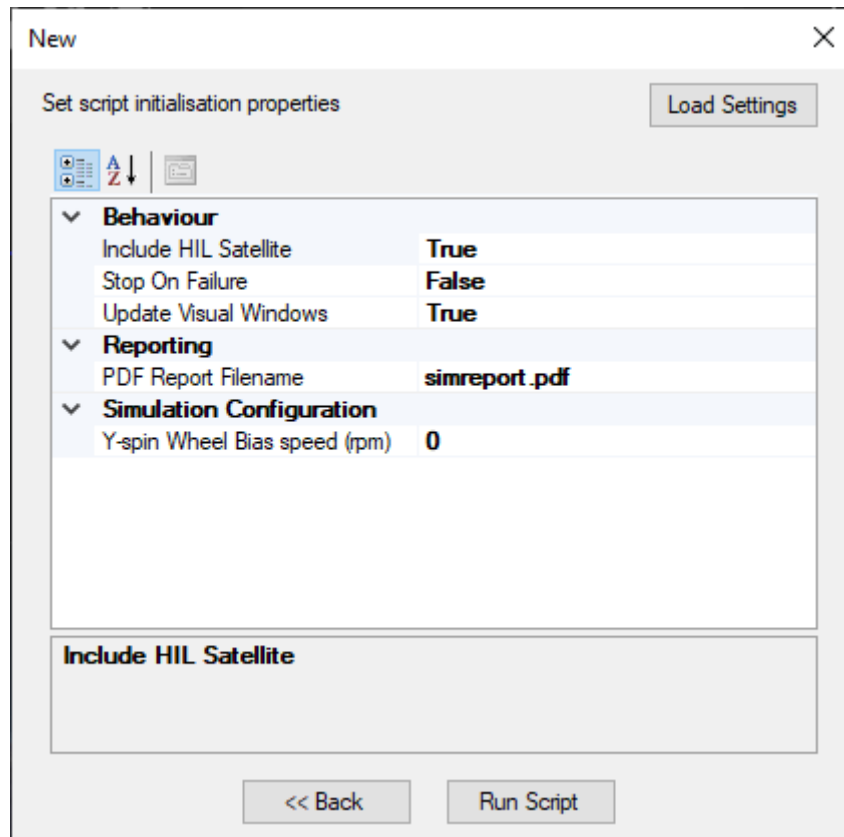


Figure 13 Script options window

After selecting a script, and pressing the “Next >>” button, the script options will be presented. The script is started by pressing the “Run Script” button. While the script is executing, the script output window will display the script progress and status messages. The script can be cancelled by pressing

the *Reset Simulation*  button.

Note that the verification script is dependent on the ADCS hardware. For 1st generation ADCS unit, the “ADCS1 Standard Verification Script” should be executed, and the ADCS2 script for 2nd generation ADCS units.

7.1 Standard simulation script and report

A built-in test script allows for the ADCS modes to be automatically changed to follow a set simulation sequence. The purpose of the script is to verify the functioning of the primary estimation and control modes for the given satellite configuration.

The script will perform two sequences. The first sequence will place the satellite in an initial tumbling motion, and then command the satellite into Y-spin mode, followed by Y-momentum stabilized mode.

An exception to this is the ADCS2 verification script, which will simulate a detumbling transition to three-axis stabilized without intermediate Y-momentum control – for pyramid reaction wheel configurations only.

The second sequence will place the satellite in three-axis controlled mode, and verify the control performance of three-axis sun-pointing and target-tracking modes. The estimation and control performance is measured throughout the script execution and reported in the script output window.

At the end of the script, a PDF report will be generated with the script results. The specific sequence that is simulated is documented in the generated report along with the individual test pass or fail results.

7.2 Scheduler Script

The Scheduler Script allows for a “programmable” sequence to be simulated. The sequence in which control or estimation modes are activated can be set by the user as part of the script properties. The sequence is defined by specifying a list of time-dependent property/value pairs. Although it is possible to enter the sequence from the script options window (Figure 13), it is more convenient to maintain the sequence in a text XML file. From the script options window, it is possible to load the predetermined script options by pressing the “Load Settings” button and selecting the XML file.

An example XML file for the Scheduler Script, for an ADCS1 unit, is given below.

```
<?xml version="1.0" encoding="utf-8"?>
<SchedulerScript xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <UpdateVisual>true</UpdateVisual>
  <ScheduledSettings>
    <ScheduledPropertySet Time="0" ObjectId="CubeACP"
Property="EstimationMode" Value="None" />
    <ScheduledPropertySet Time="0" ObjectId="CubeACP (HIL)"
Property="EstimationMode" Value="None" />
    <ScheduledPropertySet Time="0" ObjectId="CubeACP"
Property="ControlMode" Value="None" />
    <ScheduledPropertySet Time="0" ObjectId="CubeACP (HIL)"
Property="ControlMode" Value="None" />
    <ScheduledPropertySet Time="1000" ObjectId="CubeACP"
Property="EstimationMode" Value="EkfFullState" />
    <ScheduledPropertySet Time="1000" ObjectId="CubeACP (HIL)"
Property="EstimationMode" Value="EkfFullState" />
    <ScheduledPropertySet Time="1000" ObjectId="CubeACP"
Property="ControlMode" Value="XyzWheel" />
    <ScheduledPropertySet Time="1000" ObjectId="CubeACP (HIL)"
Property="ControlMode" Value="XyzWheel" />
    <ScheduledPropertySet Time="1500" ObjectId="CubeACP"
Property="AttitudeReference" Value="{Roll:0, Pitch:-15, Yaw:0}" />
    <ScheduledPropertySet Time="1600" ObjectId="CubeACP"
Property="AttitudeReference" Value="{Roll:0, Pitch:0, Yaw:0}" />
    <ScheduledPropertySet Time="2000" ObjectId="CubeACP"
Property="TrackingTarget" Value="{LongRef:90, LatRef:25, AltRef:10}" />
    <ScheduledPropertySet Time="2200" ObjectId="CubeACP"
Property="ControlMode" Value="RwTargetTrack" />
    <ScheduledPropertySet Time="2500" ObjectId="CubeACP"
Property="ControlMode" Value="XyzWheel" />
  </ScheduledSettings>
  <SaveState>true</SaveState>
  <CsvFilename>output.csv</CsvFilename>
  <StateSaveInterval>10</StateSaveInterval>
  <SaveIgnoreObjects>Magnetometer (HIL) ,Magnetorquer (HIL) ,Reaction
Wheels (HIL) ,CubeSense1 (HIL) ,CubeSense2 (HIL) ,CSS (HIL) ,Rate
Sensor (HIL) ,CubeACP (HIL) ,Solar Panel1 (HIL) ,Solar Panel2 (HIL) ,Solar
Panel3 (HIL) ,Solar Panel4 (HIL) ,Solar Panel5 (HIL) ,Solar Panel6 (HIL) ,Solar
Panel7 (HIL) ,Solar Panel8 (HIL) ,Solar Panel9 (HIL)</SaveIgnoreObjects>
</SchedulerScript>
```

An equivalent Scheduler script XML file for an ADCS2 is given next:

```

<?xml version="1.0" encoding="utf-8"?>
<SchedulerScript xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <UpdateVisual>true</UpdateVisual>
  <ScheduledSettings>
    <ScheduledPropertySet Time="0" ObjectId="CubeComputer"
Property="EstimationMode" Value="EstNone" />
    <ScheduledPropertySet Time="0" ObjectId="CubeComputer (HIL) "
Property="EstimationMode" Value="EstNone" />
    <ScheduledPropertySet Time="0" ObjectId="CubeComputer"
Property="ControlMode" Value="ConNone" />
    <ScheduledPropertySet Time="0" ObjectId="CubeComputer (HIL) "
Property="ControlMode" Value="ConNone" />
    <ScheduledPropertySet Time="1000" ObjectId="CubeComputer"
Property="EstimationMode" Value="EstFullEkf" />
    <ScheduledPropertySet Time="1000" ObjectId="CubeComputer (HIL) "
Property="EstimationMode" Value="EstFullEkf" />
    <ScheduledPropertySet Time="1000" ObjectId="CubeComputer"
Property="ControlMode" Value="ConXYZwheel" />
    <ScheduledPropertySet Time="1000" ObjectId="CubeComputer (HIL) "
Property="ControlMode" Value="ConXYZwheel" />
    <ScheduledPropertySet Time="1500" ObjectId="CubeComputer"
Property="ReferenceCommands.CmdRpyPitch" Value="-15" />
    <ScheduledPropertySet Time="1600" ObjectId="CubeComputer"
Property="ReferenceCommands.CmdRpyPitch" Value="0" />
    <ScheduledPropertySet Time="2000" ObjectId="CubeComputer"
Property="ReferenceCommands.CmdTargetAltitude" Value="6371" />
    <ScheduledPropertySet Time="2000" ObjectId="CubeComputer"
Property="ReferenceCommands.CmdTargetLatitude" Value="-20" />
    <ScheduledPropertySet Time="2000" ObjectId="CubeComputer"
Property="ReferenceCommands.CmdTargetLongitude" Value="10" />
    <ScheduledPropertySet Time="2200" ObjectId="CubeComputer"
Property="ControlMode" Value="ConTgtTrack" />
    <ScheduledPropertySet Time="2500" ObjectId="CubeComputer"
Property="ControlMode" Value="ConXYZwheel" />
  </ScheduledSettings>
  <SaveState>true</SaveState>
  <CsvFilename>output.csv</CsvFilename>
  <StateSaveInterval>10</StateSaveInterval>
  <SaveIgnoreObjects>Magnetometer (HIL) ,Magnetorquer (HIL) ,Rate
Sensor (HIL) ,CubeComputer (HIL)</SaveIgnoreObjects>
</SchedulerScript>

```

Each entry in the schedule sequence has to specify the following four values:

- **Time:** the number of seconds since the start of the simulation
- **ObjectId:** Has to be either CubeACP or CubeACP(HIL) (ADCS Gen 1) or CubeComputer or CubeComputer(HIL) (ADCS Gen 2). If you want both the reference and HIL ADCS to execute the same commands or mode changes, you have to supply individual entries for both with the same time
- **Property:** The property to set. This can be any of the CubeACP or the CubeComputer properties (see below)
- **Value:** The value to set, in string representation.

For an ADCS1 CubeACP, the formatting of values and allowed values are described in the table below.

Property	Value string format / possible values	Example
ControlMode attitude control mode	None BDot YSpin YWheelInit XyzWheel RwSunTrack RwTargetTrack VeryFastDetumbling FastDetumbling Special1 Special2 RwStop User RWSunRollYawSteer RWTargetYawSteer	-
EstimationMode attitude estimation mode	None MemsRate MagnetometerRKF MagRatePitchRkf Triad EkfFullState EkfGyro	-
Asgp4Mode Augmented SGP4 mode	None Trigger Background Augment	-
AttitudeReference reference roll, pitch and yaw angles, to which attitude is controlled in XyzWheel control mode	{Roll:<roll>, Pitch:<pitch>, Yaw:<yaw>}	{Roll:0, Pitch:-15, Yaw:0}
TrackingTarget reference latitude, longitude and altitude to which satellite attitude is controlled to in RwTargetTrack control mode	{LongRef:<longitude>, LatRef:<latitude>, AltRef:<altitude>}	{LongRef:90, LatRef:25, AltRef:10}
WheelSpeedCmd Set wheel speeds (only applies if the control mode is not	{X:<x speed>, Y:<y speed>, Z:<z speed>}	{X:0, Y:-1000, Z:0}
InertialPointingReference Reference vector to which the attitude is controlled in special inertial-pointing control mode	{X:<x>, Y:<y>, Z:<z>}	{X:0.707, Y:0.707, Z:0}
Config.X	Depends on sub-property	Property="Config.TargetTrackFacet" Value="PosZ"

Adjust sub-property within the ACP ADCS configuration structure		
OrbitParams.X Adjust sub-property within the orbit that the ACP is using. (note, this does not change the orbit that is being simulated, only the orbit that the ACP is using for its internal model calculations).	Depends on sub-property	Property="OrbitParams.Raan" Value="90"

For ADCS2, the following properties of the CubeComputer model object can be specified with matching values and formatting.

Property	Value string format / possible values	Example
ControlMode attitude control mode	ConNone ConBDot ConYSpin ConBdot3 ConDetumble ConSunYspin ConZspin ConSunZspin ConGGboom ConGGsun ConYwheelInit ConYwheel ConXYZwheel ConSunTrack ConTgtTrack ConTgtSteer ConGndTrack ConIrcTrack ConMoonTrack ConSatTrack ConYawSun ConYawTarget ConRollSun ConRollTarget ConYawTargetBest ConStopRW ConUser	-
EstimationMode attitude estimation mode	EstNone EstGyro EstMagRkf EstPitchRkf EstTriad EstFullEkf	-

	EstGyroEkf EstUser	
ReferenceCommands.X reference roll, pitch and yaw angles, to which attitude is controlled in ConXYZwheel control mode, target latitude longitude and radius for target tracking control modes, and reference inertial pointing vector	Depends on sub-property	Property="ReferenceCommands.CmdRpyRoll" Value="-30"
OpenLoopWheelSpeeds.Rw<num>SpeedCmd Set open-loop wheel speeds	Wheel speed	Property="OpenLoopWheelSpeeds.Rw0SpeedCmd" Value="1500"
ControllerConfig.X Adjust sub-property within the controller configuration structure	Depends on sub-property	Property="ControllerConfig.Kd1" Value="0.04"
EstimatorConfig.X Adjust sub-property within the estimator configuration structure	Depends on sub-property	Property="EstimatorConfig.EkfUseFss" Value="True"
SatelliteConfig.X Adjust sub-property within the satellite configuration structure	Depends on sub-property	Property="SatelliteConfig.TgtTrackBodyVecX" Value="1"
MountConfig.X Adjust sub-property within the mounting configuration structure	Depends on sub-property	Property="MountConfig.MountFss1Alpha" Value="90"
MagMainConfig.X Adjust sub-property within the main magnetometer configuration structure	Depends on sub-property	Property="MagMainConfig.Offset1" Value="10.23"
MagRedConfig.X Adjust sub-property within the redundant magnetometer configuration structure	Depends on sub-property	Property="MagRedConfig.Offset1" Value="10.23"
NodeSelecionConfig.X	Depends on sub-property	Property="NodeSelecionConfig.SelectFss" Value="3"

Adjust sub-property within the node selection configuration structure		
RwConfig.X Adjust sub-property within the reaction wheel configuration structure	Depends on sub-property	Property="RwConfig.PyrHnom" Value="0.0025"
MtqConfig.X Adjust sub-property within the magnetorquer configuration structure	Depends on sub-property	Property="MtqConfig.OnTimeMax" Value="800"
OrbitSatParams.X Adjust sub-property within the orbit that the Control Program is using. (note, this does not change the orbit that is being simulated, only the orbit that the control program is using for its internal model calculations).	Depends on sub-property	Property="OrbitSatParams.OrbitRaan" Value="90"
OrbitTargetParams.X Adjust sub-property within the orbit that defines the reference target when using the ConSatTrack control mode	Depends on sub-property	Property="OrbitTargetParams.OrbitRaan" Value="90"

It is also possible to export the state of the simulation at every simulation time step (or at intervals that are integer multiples of the simulation time step). This is done by specifying a value of *True* for the *SaveState* parameter in the script parameters XML file (or in the script options window). The *CsvFilename* parameter has to be specified in this case. It is also possible to limit the size of the data file by specifying a comma separated list of model object for which the state should not be saved, in the *SaveIgnoreObjects* parameter.

8 Command-line and “Headless” Mode

EOS can be used in a “headless” mode where it does not instantiate a GUI window. In this case, all interaction is with the Windows command prompt. This is useful for executing the built-in scripts in an automated setup.

The following command line options are supported:

```
--version,      -v : outputs version information
--projectfile,  -p : specifies the name of the .eos project to load at startup
--scriptname,   -s : specifies the name of the script to run
--scriptparams, -o : specifies the name of the XML file file script parameters
```

The script name can be one of the following:

- Eos.Models.Satellite.CubeSpaceLibrary.Adcs1StandardVerificationScript
- Eos.Models.Satellite.CubeSpaceLibrary.Adcs2StandardVerificationScript
- Eos.Models.Satellite.CubeSpaceLibrary.SchedulerScript

Example usage is given below

```
C:\>Eos.exe      --projectfile="C:\Project      Directory\myproject.eos"      --
scriptname=Eos.Models.Satellite.CubeSpaceLibrary.Adcs1StandardVerificationScript --
scriptparams="C:\Project Directory\hilttest.xml"

C:\>Eos.exe -pmyproj.eos -sStandardVerificationScript
```

The script output, which would normally appear in the script output window in the EOS GUI, will now be printed to the console window.

Appendix A Simulation Model Overview

The CubeSat ADCS simulation will simulate the attitude dynamics of a CubeSat in a given orbit. The attitude of the satellite is integrated numerically, taking into account the following torques

- Control torque from reaction wheels and magnetorquers
- Disturbance torques
 - o Gravity gradient
 - o Aerodynamic
 - o Solar radiation pressure
 - o Magnetic torque from solar panel induced currents

The orbit of the satellite is calculated using an SGP4 orbit propagator. The mean orbital elements can be adjusted by the user, by selecting the Orbit Model object and adjusting the properties for the selected object, as described in 5.2 and 5.3.

The simulation model state is updated at fixed time intervals. This update interval can be changed, as specified in 5.7. At each interval, the orbital state (position and velocity) is calculated from the SGP4 propagator. The disturbance torques are calculated for the given attitude state (attitude quaternion and angular velocity), and for the given orbital position.

The disturbance and control torques, and also the sensor measurements, require calculation of a number of environmental parameters that are dependent on current time and position of the satellite. The environmental state of the satellite includes the magnetic field strength and direction, atmospheric density and relative sun direction. The environmental parameters are calculated using environmental models for the magnetic field (IGRF), atmosphere (NRLMSISE-00) and a sun orbital model.

The control torques are calculated by simulating the satellite control system in detail. Firstly, the sensor measurements (magnetometer, sun and nadir sensor, star sensor and rate sensor) are simulated by taking the true values and adding suitable sensor noise. These are supplied to the Attitude Control Processor model, which represents the control program that runs on the CubeComputer. The Attitude Control Processor will use the supplied sensor inputs to calculate a commanded wheel speed and magnetorquer pulse command. These latter commands are supplied to the actuator models – one for the three axis reaction wheels, and one for the magnetorquers. These latter actuator models will then produce the control torque that the satellite control system is effecting.

The control torque, together with the disturbance torques are summed and used in the Euler dynamics equation to produce an angular acceleration vector. The angular acceleration is then numerically integrated, using a 4th order Runge-Kutta numerical integration scheme, to find the angular velocity and attitude quaternion for the next simulation iteration.

This procedure is applied for two separate satellite objects – one is the reference satellite in which the Attitude Control Processor is implemented as a representative model of the actual CubeComputer control program. The second satellite has a different Attitude Control Processor model implementation – one that communicates using the USB GSE cable, or PCAN USB adapter, to the actual CubeComputer – to perform a Hardware-in-the-Loop (HIL) simulation.

Appendix B Coordinate Systems

The Earth-Centred Inertial (ECI) coordinate frame has its origin at the Earth geometric centre. The Z axis is aligned with the Earth rotation axis and the X and Z axes are defined by celestial bodies. The ECEF (Earth Centred, Earth Fixes) coordinate system rotates with the Earth. The Z-axis is again aligned with the rotation axis, so that a transform between ECI and ECEF coordinates simply involve a rotation about the Z axis. The rotation angle is dependent on the GMST (Greenwich Mean Sidereal Time).

The satellite attitude is propagated in the orbit referenced coordinate (ORC) system. The orbit coordinate system rotates with the orbit so that the $\mathbf{Z}_o$ reference axis points towards nadir, the $\mathbf{X}_o$ reference axis points towards the velocity vector and the $\mathbf{Y}_o$ axis points along the orbit anti-normal. The satellite coordinate system will nominally be aligned with the orbit coordinate system when roll, pitch and yaw equal 0.

The Earth-centred inertial coordinate frame (ECI) is used to propagate the position of the satellite and third bodies. A transformation is required to convert from the inertial frame, repositioned to the satellite current position, into the orbit coordinate frames. This transformation can be obtained from the satellite position, $\mathbf{r}$, and velocity, $\mathbf{v}$, vector.

$$\mathbf{A}_{I/o} = \begin{bmatrix} (\mathbf{r} \times (\mathbf{v} \times \mathbf{r}))^T \\ (\mathbf{v} \times \mathbf{r})^T \\ -\mathbf{r}^T \end{bmatrix} \quad 8-1$$

The orbit angular rate, ω_o , is used to convert angular rates in the orbit reference frame to the inertial frame and for a circular orbit it is given by:

$$\omega_o = \frac{|\mathbf{v}|}{|\mathbf{r}|} \quad 8-2$$

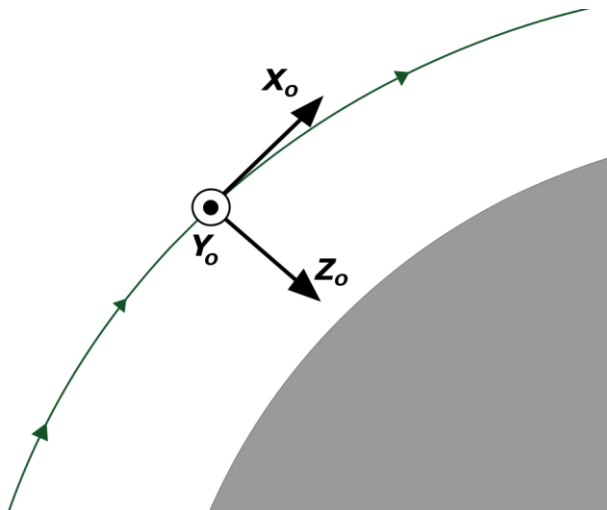


Figure 14 Orbit Reference Frame

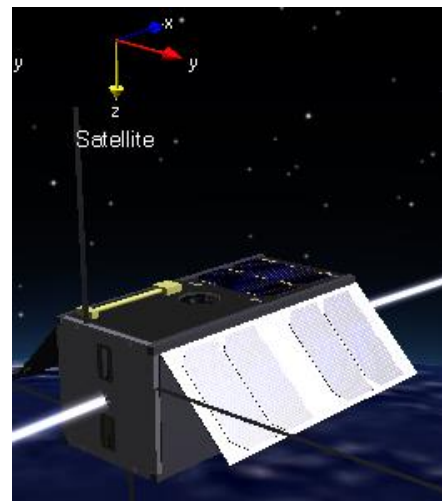


Figure 15 Satellite Body Coordinate Frame

Appendix C Model Object Detail

The specific implementation, adjustable properties and output state of each model block is discussed in this section.

Model Object Name	Simulation Root
Detail	Acts as a container in which an Earth orbiting satellite simulation runs. This object has a method called 'Update' which is called at every iteration of the simulation.
Adjustable Properties	None

Model Object Name	Environment
Detail	Acts as a container for all the environmental models
Adjustable Properties	None
State	None

Model Object Name	Sun Model
Detail	Provides the sun's position in ECI frame as a function of time. Implementation as in [2] pp.151-153
Adjustable Properties	None
State	None

Model Object Name	Moon Model
Detail	Provides the moon's position in ECI frame as a function of time. Implementation as in [2] pp.307-308
Adjustable Properties	None
State	None

Model Object Name	Atmosphere Model
Detail	Provides the atmosphere density for a given time and ECI position vector, using the NRLMSISE-00 atmosphere model [3]
Adjustable Properties	None
State	None

Model Object Name	Solar and Geo Index Provider
Detail	Provides Ap and F10.7 solar index to the NRLMSISE-00 model. For historic dates, Ap and F10.7 indices are found from a data file. For future dates, the indices are calculated from a 'mean' 11-year profile, based on [4]. Historic daily values are available, but prediction is only based on monthly intervals.

Adjustable Properties	Solar and Geomagnetic Mode	Selects how geomagnetic index and solar flux is found.	
		HistoricDailyThen-PredicatedMonthly	Returns daily data from data file as long as daily observations are available. For future dates predicted monthly vales are returned
		HistoricAndPredicted-Monthly	Returns historic and predicted values based on monthly intervals only
		MeanMonthlyOnly	Returns only mean 11-year cycle values
		Constant	Returns the supplied input properties (Constant Solar Flux and Constant Geomagnetic (Ap) Index)
	Constant Solar Flux	Constant F10.7 solar flux to be used if the ‘Solar and Geomagnetic Mode’ is set to ‘Constant’	
Constant Geomagnetic (Ap) Index	Constant Ap geomagnetic index to be used if the ‘Solar and Geomagnetic Mode’ is set to ‘Constant’		
State	None		

Model Object Name	Magnetic Field Model
Detail	Provides the magnetic field vector (size and direction) at the supplied input position, using the International Geomagnetic Field Reference Model (IGRF) [5]
Adjustable Properties	None
State	None

Model Object Name	Satellite Model <Sat Name>	
Detail	Reference satellite model.	
Adjustable Properties	+ Physical Properties	
	Mass	The mass of the satellite in kg units
	Moment of Inertia	The moment of inertia tensor for the satellite in kg.m ² units $\mathbf{I} = \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \\ M_{31} & M_{32} & M_{33} \end{bmatrix}$
State	+ External Disturbances	

	Aerodynamic Force	Total aerodynamic force calculated from the satellite surface model (relative to satellite body coordinate frame)
	Aerodynamic Torque	Total aerodynamic torque calculated from the satellite surface model (relative to satellite body coordinate frame)
	Gravity Gradient Torque	Gravity gradient torque
	Solar Force	Total solar force calculated from the satellite surface model (relative to satellite body coordinate frame)
	Solar Torque	Total solar torque calculated from the satellite surface model (relative to satellite body coordinate frame)
	+ State	
	Satellite Environment State	Sun position vector, magnetic field vector, atmospheric density and relative atmosphere velocity vector for the satellite at the current instant. All vectors relative to the satellite body coordinate frame.
	Satellite Kinematic State	Orbital position and velocity in ECI frame, angular velocity vector and attitude quaternion at the current instant

Model Object Name	Kinematic Model
Detail	Provides orbit position and velocity, and attitude angles and angular rates to the root simulation object. This model implements a numerically integrated attitude, where position and velocity is supplied by an SGP4 orbit model. The spacecraft attitude is the transform that describes the orientation of the spacecraft body frame relative to the orbit reference frame. This transform is modelled using a quaternion representation, $\mathbf{q} = [q_1 \ q_2 \ q_3 \ q_4]^T$, to avoid singularities. The attitude quaternion is also expressed as a direction cosine matrix, $\mathbf{A}_{O \rightarrow B} = \begin{bmatrix} q_1^2 - q_2^2 - q_3^2 + q_4^2 & 2(q_1q_2 + q_3q_4) & 2(q_1q_3 - q_2q_4) \\ 2(q_1q_2 - q_3q_4) & -q_1^2 + q_2^2 - q_3^2 + q_4^2 & 2(q_2q_3 + q_1q_4) \\ 2(q_1q_3 + q_2q_4) & 2(q_2q_3 - q_1q_4) & -q_1^2 - q_2^2 + q_3^2 + q_4^2 \end{bmatrix} \quad [1]$ and as roll(θ), pitch(ϕ) and yaw(ψ) Euler angles. The latter makes use of a 2-1-3 Euler rotation sequence. The angular rates of the spacecraft body relative to the orbit coordinate frame, $\boldsymbol{\omega}_B^O = [\omega_{xo} \ \omega_{yo} \ \omega_{zo}]^T$, are used to propagate the attitude. $\dot{\mathbf{q}} = \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \\ \dot{q}_4 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 0 & \omega_{zo} & -\omega_{yo} & \omega_{xo} \\ -\omega_{zo} & 0 & \omega_{xo} & \omega_{yo} \\ \omega_{yo} & -\omega_{xo} & 0 & \omega_{zo} \\ -\omega_{xo} & -\omega_{yo} & -\omega_{zo} & 0 \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{bmatrix} \quad [2]$ The attitude dynamics of the spacecraft are described by the Euler equation $\mathbf{I}\dot{\boldsymbol{\omega}}_B^I = \mathbf{N}_{GG} + \mathbf{N}_{MT} - \dot{\mathbf{h}}_w - \boldsymbol{\omega}_B^I \times (\mathbf{I}\boldsymbol{\omega}_B^I + \mathbf{h}_w) \quad [3]$ where

	$\mathbf{I}$ is the moment of inertia tensor of the satellite, $\boldsymbol{\omega}_B^I = \boldsymbol{\omega}_B^O + \mathbf{A}_{I \rightarrow O} [0 \quad -\omega_o \quad 0]^T$ is the angular rate of the satellite relative to the ECI coordinate frame, $\mathbf{N}_{GG}$ is the gravity gradient disturbance torque, $\mathbf{N}_{MT}$ is the control torque produced by the magnetic torquer rods and, $\mathbf{h}_w = [h_{wx} \quad h_{wy} \quad h_{wz}]^T$ is the reaction wheel angular momentum vector. The gravity gradient torque is as a result of varying gravitational force on parts of the spacecraft depending on their distance to the Earth. It is calculated from the equation $\mathbf{N}_{GG} = 3\omega_o^2 (\mathbf{z}_o^B \times \mathbf{I} \mathbf{z}_o^B) \quad [4]$ where $\mathbf{z}_o^B = A_{O \rightarrow B} [0 \quad 0 \quad 1]^T$ is the nadir vector in the spacecraft body coordinate frame. The magnetic torquer rod and reaction wheel control torques are described in relevant actuator models	
Adjustable Properties	+ Initial Values	
	Initial angular rates	Initial angular rate vector (orbit referenced) in deg/s units
	Initial roll angle	Initial roll angle (deg)
	Initial pitch angle	Initial pitch angle (deg)
	Initial yaw angle	Initial yaw angle (deg)
State	None	

Model Object Name	Orbit Model	
Detail	Provides orbit position and velocity based on an SGP4 orbit propagator	
Adjustable Properties	<All>	All properties as per definition of the TLE entry data ³
State	None	

Satellite Components

Model Object Name	Antenna
Detail	Antenna object. Serves no simulation purpose, only visual
Adjustable Properties	None
State	None

Model Object Name	Magnetometer
Detail	Simulated magnetic field sensor. Takes the magnetic field vector that was produced by the satellite environment models as the true state, and adds correlated “random walk” noise, where the time-series of the noise per each of the X, Y and Z channels are obtained from $n_{x,k} = (1 - \alpha)n_{x,k-1} + \alpha A(\Omega_{x,k} - 0.5)$

³ <https://www.celestrak.com/NORAD/documentation/tle-fmt.asp>

	Where α is the smoothing factor or time constant of the filter, and Ω_k is a number from a pseudo-random sequence (uniform white noise) obtained from the built-in operating system <i>rand()</i> function – it returns a random floating point number between 0 and 1. A is the amplitude of the noise signal. The raw magnetometer measurement is then $\mathbf{B}_{raw} = \begin{bmatrix} B_{IGRF,x} + n_{x,k} \\ B_{IGRF,y} + n_{y,k} \\ B_{IGRF,z} + n_{z,k} \end{bmatrix}$ Where $\mathbf{B}_{IGRF} = [B_{IGRF,x} \ B_{IGRF,y} \ B_{IGRF,z}]^T$ is the true magnetic field vector and $n_{x,k}$, $n_{y,k}$ and $n_{z,k}$ is the additive noise. A is set to 5.0E-7. The value that is used for α is 0.04. This results in an RMS error per channel of 20 nT. The model then further applies a sensitivity and offset error, such that the ACP calibration should correct for it (this simulates the calibration correction that is needed for true magnetometer hardware). The calibration transformation that takes place on the magnetometer is given by $\mathbf{B}_{raw,calib} = \mathbf{S}[\mathbf{M}_{mag_mount}\mathbf{B}_{raw} + \mathbf{d}]$ Where $\mathbf{M}_{mounting}$ is the mounting transformation DCM. $\mathbf{B}_{raw}$, is the measured magnetic field, (true magnetic field vector in the body coordinate frame, with noise added) and $\mathbf{B}_{raw,calib} = [B_{raw_x} \ B_{raw_y} \ B_{raw_z}]^T$ is the raw measurement returned by the CubeControl hardware, that includes the calibration transformation that has to be corrected in the ACP.	
Adjustable Properties	+ Calibration	
	Inverse Sensitivity	Inverse Sensitivity matrix $\mathbf{S}$ in the above equation
	Inverse Offset	Inverse Offset vector $\mathbf{d}$ in the above equation
	+ State	
	Deployed	Flag to control the stowed/deployed state of the magnetometer.
State	+ Measurement	
	Measured Magnetic Field Vector	Measured magnetic field vector in nT units
	Measurement Vector Angle Error	Error between true magnetic field vector and measurement vector direction
	Measured Vector Length Error	Error between true magnetic field vector and measurement vector length
	Raw ADC Measurement	$\mathbf{B}_{raw,calib}$ in the above equation

Model Object Name	Magnetorquer
Detail	Simulated magnetic torque rods. The 3-axis torquer rods will produce a torque $\mathbf{N}_{MT} = M_{max}\mathbf{M}_{PWM} \times \mathbf{B}_{IGRF}$ Where $\mathbf{B}_{IGRF}$ is the true magnetic field vector, supplied by the IGRF model, and $\mathbf{M}_{PWM}$ is the vector with pulse durations for each of the X, Y and Z axes as commanded by the control system. $\mathbf{M}_{PWM}$ is the PWM duty cycle of the pulse. Normally the magnetic control loop runs at 1Hz, and $\mathbf{M}_{PWM}$ is limited to 80% or 0.8s. The PWM command is further limited to 0.01s resolution.

	M_{max} is the maximum magnetic dipole moment for the torquer rod.	
Adjustable Properties	+ Physical	
	Position	Position vector of magnetorquer assembly (CubeCoil and Y and Z rods)
State	+ Command	
	Pulse duration	The $\mathbf{M}_{PWM}$ vector in the above equation
	+ State	
	Control Torque Vector	The torque vector, $\mathbf{N}_{MT}$, produced by the torquer rods (in body coordinates)
	Magnetic dipole vector	Effective magnetic dipole moment for the current PWM command

Model Object Name	Reaction Wheels (ADCS Gen 1)	
Detail	Simulated 3-axis reaction wheels The input to the reaction wheel model is the commanded wheel torque, $\mathbf{N}_w$. The reaction wheel model will saturate the input torque so that it is below the maximum wheel torque, N_{max}. The commanded wheel momentum is then $\mathbf{h}_{com} = \mathbf{h}_w + \text{sat}(\mathbf{N}_w, N_{max}) \quad [1]$ where $\mathbf{h}_w$ is the current wheel momentum. The saturation function is then again applied to the commanded wheel momentum to limit the maximum wheel momentum (and thus wheel speed) to h_{max}. The wheel speed and angular momentum of the reaction wheel is related by the wheel disc inertia such that $\mathbf{h}_w = I_w \boldsymbol{\omega}$ Where $\boldsymbol{\omega}$ is the vector with wheel speed and I_w is the disc inertia.	
Adjustable Properties	None	
State	+ Command	
	Speed command	The wheel speed command issued by the ACP
	+ Measurement	
	Measured Angular Momentum	Measured angular momentum
	Measured Speed	Measured wheel speed
	+ State	
	Angular moment vector	The angular moment vector of the three axis reaction wheels, $\mathbf{h}_w$
	Control torque vector	Time derivative of the angular moment vector, $\dot{\mathbf{h}}_w$

Model Object Name	Reaction Wheel (ADCS Gen 2)	
Detail	Simulated single axis reaction wheel The input to the reaction wheel model is the commanded wheel torque, N_w. The reaction wheel model will saturate the input torque so that it is below the maximum wheel torque, N_{max}. The commanded wheel momentum is then	

	$h_{com} = h_w + \text{sat}(N_w, N_{max}) \quad [1]$ where h_w is the current wheel momentum. The saturation function is then again applied to the commanded wheel momentum to limit the maximum wheel momentum (and thus wheel speed) to h_{max}. The wheel speed and angular momentum of the reaction wheel is related by the wheel disc inertia such that $h_w = I_w \omega$ Where ω is the wheel speed and I_w is the disc inertia.	
Adjustable Properties	None	
State	+ Command	
	Speed command	The wheel speed command issued by the CubeComputer
	+ Measurement	
	Measured Angular Momentum	Measured angular momentum
	Measured Speed	Measured wheel speed
	+ State	
	Angular moment vector	The angular moment of the reaction wheels, h_w
	Control torque vector	Time derivative of the angular moment, $\dot{h}_w$

Model Object Name	CubeSense (1 or 2)	
Detail	Simulated sun and/or nadir sensor. This model uses the sun vector returned by the Sun environment model and the position returned by the Orbit Model as true state for the sun and nadir sensors. A random rotation of the measurement vector is then applied to simulate sensor noise.	
Adjustable Properties	None	
State	+ Nadir	
	Measured Nadir Vector	Measured nadir vector relative to nadir sensor mounting
	Nadir Angle Measurement Error	Angle error between true and measured nadir vector
	+ Sun	
	Measured Sun Vector	Measured sun vector relative to sun sensor mounting
	Sun Angle Measurement Error	Angle error between true and measured sun vector

Model Object Name	CSS	
Detail	Simulated Coarse Sun Sensor (photodiodes). This model uses the sun vector returned by the Sun environment model. A random rotation of the measurement vector is then applied to simulate sensor noise.	

Adjustable Properties	None	
State	+ Measurement	
	Measured Sun Vector	Measured sun vector relative to satellite body
	Sun Angle Measurement Error	Angle error between true and measured sun vector

Model Object Name	Rate Sensor	
Detail	Simulated MEMS rate sensor. Takes the true angular rate vector as the true state, and adds a correlated “random walk” bias offset, where the time-series of the bias per each of the X, Y and Z channels are obtained from $n_{x,k} = \beta n_{x,k-1} + \alpha(\Omega_{x,k} - 0.5)$ Where Ω_k is a number from a pseudo-random sequence (uniform white noise) obtained from the built-in operating system <i>rand()</i> function – it returns a random floating point number between 0 and 1. The rate sensor measurement is then $\omega_{meas} = \omega_{o/b} + [n_{x,k} \ n_{y,k} \ n_{z,k}]^T + \gamma[p_{x,k} \ p_{y,k} \ p_{z,k}]^T$ Where $p_{x,k}$, $p_{y,k}$ and $p_{z,k}$ are random ‘white’ noise discrete time series.	
Adjustable Properties	None	
State	+ Measurement	
	Bias	The current bias offset vector, $[n_{x,k} \ n_{y,k} \ n_{z,k}]^T$
	Measured Angular Rate	The measured angular rate, ω_{meas}

Model Object Name	CubeACP (ADCS Gen1 only)	
Detail	Attitude Control Processor - this represents the CubeComputer in the CubeSpace ADCS bundle. For details about the implementation and adjustable properties, please consult the CubeACP Reference Manual ⁴	
Adjustable Properties	+ Actuator Commands	
	Wheel Speed Reference	Open-loop wheel speed command
	+ Configuration	
	ACP Configuration	ACP Configuration structure (see CubeACP Reference Manual for detail)
	Orbit Parameters	TLEs used by the CubeACP
	+ Mode	
	Selected Control Mode	Selected control mode (see CubeACP Reference Manual for detail)
	Selected Estimation Mode	Selected estimation mode (see CubeACP Reference Manual for detail)

⁴ <https://cubespace.co.za/>

	Selected ASGP4 Mode	Selected mode for augmented SGP4 operation (SGP4 orbit elements augmented using GPS measurements)
	+ Pointing Selection	
	Attitude Reference	Reference roll, pitch and yaw angles for XYW Wheel mode
	Tracking Parameters	Reference latitude, longitude and altitude for target in XYZ Wheel target tracking mode
State	+ ACP State	
	AdcsState	Current ADCS run mode
	ControlModeChangeNotAllowed	Flag to indicate error in control mode change request
	ControlModeState	Current control mode
	EstimationModeState	Current estimation mode
	EstimatorChangeNotAllowed	Flag to indicate error in estimation mode change request
	+ Error	
	Estimated Angular Rate Error	The error between true and estimated angular rate
	Estimated Roll/Pitch/Yaw Error	The error between true and estimated attitude angles
	+ Estimated State	
	Estimated Kinematic State	Estimated orbit position and velocity (from SGP4 orbit propagator running in CubeACP), estimated angular velocity vector and estimated attitude quaternion/angles

Model Object Name	CubeComputer (ADCS Gen2 only)	
Detail	This represents the CubeComputer in the CubeSpace ADCS bundle.	
Adjustable Properties	+ Actuator Commands	
	Open-loop Wheel Commands	Open-loop wheel speed command
	+ Configuration	
	Controller Configuration	Controller configuration settings
	Estimator Configuration	Estimator configuration settings
	Magnetorquer Configuration	Magnetorquer configuration settings
	Main Magnetometer Calibration	Main magnetometer calibration settings
	Mounting Configuration	Mounting configuration settings
	Node Selection Configuration	Node selection configuration settings
	Reaction Wheel Configuration	Reaction wheel configuration settings
	Redundant Magnetometer Calibration	Redundant magnetometer calibration settings
	Satellite Configuration	Satellite configuration settings
	Satellite Orbit Parameters	TLEs used by the control program to estimate satellite position
	Target Orbit Parameters	TLEs used to provide reference position for target tracking, when using the ConSatTrack control mode
	+ Mode	

	Selected Control Mode	Selected control mode
	Selected Estimation Mode	Selected estimation mode
	+ Controller Reference Commands	
	Reference Commands	Reference roll, pitch and yaw angles for XYW Wheel mode, target tracking reference latitude, longitude and radius, and inertial pointing reference vector
State	+ Current Mode	
	Control Mode	Current control mode
	Estimation Mode	Current estimation mode
	+ Error	
	Estimated Angular Rate Error	The error between true and estimated angular rate
	Estimated Roll/Pitch/Yaw Error	The error between true and estimated attitude angles
	+ Estimated State	
	Estimated Kinematic State	Estimated orbit position and velocity, estimated angular velocity vector and estimated attitude quaternion/angles

Model Object Name	Panel / Solar Panel								
Detail	Simulated satellite surfaces. The panel and solar panel objects define surfaces such that the torque as a result of aerodynamic and SRP forces are evaluated per surface segment. Per segment i, the aerodynamic force is calculated from [6] $\mathbf{F}_{aero,i} = \rho \mathbf{v}_{rel} ^2 A_i \cos \theta [\sigma_{t,i} \hat{\mathbf{v}}_{rel} + (\sigma_{n,i} S_i + (2 - \sigma_{n,i} - \sigma_{t,i}) \cos \theta) \mathbf{n}_i]$ Where ρ is the atmospheric density, A_i is the area of the surface, $\mathbf{v}_{rel}$ is the velocity of the satellite relative to the atmosphere $\hat{\mathbf{v}}_{rel}$ is the unit vector representing only the direction of the incoming gas molecules $\mathbf{n}_i$ is the normal vector to the surface $\theta_i = \cos^{-1}(\hat{\mathbf{v}}_{rel} \cdot \mathbf{n}_i)$ is the angle between the surface normal and the incident gas stream $\sigma_{t,i}$ is the tangential accommodation coefficient, $\sigma_{n,i}$ is the normal accommodation coefficient, S_i is the ratio of molecular exit velocity to incoming velocity. The molecular exit velocity depends on the type of gas. The normal and tangential accommodation coefficients are also dependent on the gas composition. The following constant values are used [7]: <i>Table 1 Typical surface properties used in aerodynamic force model</i> <table> <tr> <th>Constants</th><th>Value</th></tr> <tr> <td>σ_n</td><td>0.8</td></tr> <tr> <td>σ_t</td><td>0.8</td></tr> <tr> <td>S</td><td>0.05</td></tr> </table>	Constants	Value	σ_n	0.8	σ_t	0.8	S	0.05
Constants	Value								
σ_n	0.8								
σ_t	0.8								
S	0.05								

The torque as a result of the aerodynamic force on panel i is then given by

$$\mathbf{N}_{aero,i} = \mathbf{c}_i \times \mathbf{F}_{aero,i}$$

Where

$\mathbf{c}_i$ is the vector from the centre of mass of the satellite to the centre of pressure of segment i , The centre of pressure is typically located in the geometric centre if rectangular surface segments are used.

The solar force on segment i is given by [8]

$$\mathbf{F}_{solar,i} = PA_i(\mathbf{s} \cdot \mathbf{n}_i) \left\{ (1 - \rho_{s,i} - \rho_{t,i})\mathbf{s} + \left[2\rho_{s,i}(\mathbf{s} \cdot \mathbf{n}_i) + \frac{2}{3}\rho_{d,i} \right] \mathbf{n}_i \right\}$$

$P = 4.563 \times 10^{-6} \text{ N/m}^2$ is the nominal solar-radiation-pressure constant 1 AU from the sun,

A_i , as before, is the surface area,

$\mathbf{n}_i$ is again the normal vector to the surface and

$\mathbf{s}$ is a unit vector pointing from the sun to the surface.

$\rho_{s,i}$, $\rho_{d,i}$ and $\rho_{t,i}$ represent the optical properties of the surface, and are the specularly reflected, diffusely reflected and transmitted portions of incoming photons respectively. The set of optical properties is completed by the fraction of photons that are absorbed by the surface, $\rho_{a,i}$. Together these four factors should add up to a total of 1.

The constants that are used in EOS are listed below [8]:

Table 2 Typical reflective surface properties used in solar force model

Constants	Value
ρ_s	0.83
ρ_d	0.0
ρ_t	0.0
ρ_a	0.17

As with aerodynamic torque, the torque as a result of the solar force on segment i is given by

$$\mathbf{N}_{solar,i} = \mathbf{c}_i \times \mathbf{F}_{solar,i}$$

The values A_i , $\mathbf{n}_i$, $\sigma_{t,i}$, $\sigma_{n,i}$, S_i , $\rho_{s,i}$, $\rho_{d,i}$, $\rho_{t,i}$ and $\mathbf{c}_i$ are all defined for every segment that makes up the satellite surface.

Solar panels model objects also contribute to external disturbance torque. The maximum magnetic moment that a panel can generate is listed below

Table 3 Maximum magnetic moment from solar panels

Panel size	Maximum magnetic moment (A.m ²)
1U	0.001
2U	0.002
3U	0.003
6U	0.006

The torque that the panel exerts is given by

$$\mathbf{N}_{panel} = [0 \quad 0 \quad M_{max} \cos \alpha]^T \times \mathbf{B}_{IGRF}$$

Where

	M_{max} is the maximum moment from Table 3, α is the angle between the solar panel normal and the incident sun vector. $\mathbf{B}_{IGRF}$ is the true magnetic field vector, as calculated from the IGRF magnetic field model. The torque above is calculated in the panel coordinate system, where the solar panels face in the +Z direction. The torque is then transformed to the satellite body coordinate frame, using the mounting transform for the panel.
Adjustable Properties	None
State	None

HIL satellite models

Model objects for the HIL satellite are exact copies of the reference components, except for the CubeACP (ADCS Gen 1) or CubeComputer (ADCS Gen 2). The HIL satellite has a HIL (Hardware in the Loop) CubeACP component. The HIL CubeACP model object will communicate with the actual CubeComputer and CubeACP program using a USB hardware interface. The estimation and control processing is thus performed on the remote hardware.

References

- [1] CubeSpace, “CubeADCS User Manual v7.1,” CubeSpace, 2020.
- [2] J. H. Meeus, *Astronomical Algorithms*, Willmann-Bell, Incorporated,, 1991.
- [3] J. Picone, A. Hedin, D. Drob and A. Aikin, “NRLMSISE-00 empirical model of the atmosphere: Statistical comparisons and scientific issues,” *Journal of Geophysical Research*, vol. 107, no. A12, pp. SIA 15-1–SIA 15-16, 2002.
- [4] K. O. Niehuss, W. W. Vaughan and H. C. Euler, *Statistical Technique for Intermediate and Long-range Estimation of 13-month Smoothed Solar Flux and Geomagnetic Index*, National Aeronautics and Space Administration, Marshall Space Flight Center, 1996.
- [5] Thébault, “International Geomagnetic Reference Field: the 12th generation,” *Earth, Planets and Space*, vol. 67, 2015.
- [6] NASA, “Spacecraft Aerodynamic Torques SP-8058,” NASA, Washington, 1971.
- [7] M. L. Gargasz, *Optimal Spacecraft Attitude Control Using Aerodynamic Torques*, Air Force Institute of Technology, 2007.
- [8] C. R. McInnes, *Solar Sailing: Technology, Dynamics, and Mission Applications*, Springer, 2004.